CDAC Advanced Machine Learning — Day 8

Master Technical Summary: Polynomial Regression, Model Complexity & K-Means Clustering

Module: Supervised & Unsupervised ML	**Target Exam:** CDAC ML & Practical Labs	**Date:** October 2026	**Author:** abs6187

PART 1: Polynomial Regression & Model Complexity

1.1 Why Polynomial Regression? Standard Simple Linear Regression assumes a strictly monotonic straight-line relationship ($y = mx + c$). However, most physical, biological, and engineering phenomena follow non-linear trajectories: crop yields peak and drop, heating costs drop with temperature before air conditioning costs push them back up, and engine efficiency curves form inverted bells. Polynomial regression handles these by engineering non-linear feature powers while maintaining linear parameter estimation via Ordinary Least Squares (OLS).

Mathematical Hypothesis Function:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \ldots + \beta_d x^d$$

- **Matrix Representation:** $\hat{y} = X_{poly}\,\beta$, where $X_{poly} = [1, x, x^2, \ldots, x^d]$
- **OLS Loss Function (MSE):** $J(\beta) = (1/n)\,\Sigma\,(y_i - \hat{y}_i)^2 \rightarrow$ Minimized via Normal Equation: $\beta = (X^T X)^{-1} X^T y$

1.2 Polynomial Degree & Curvature Quick Guide

Degree	Mathematical Curve	Bends	Real-World Example	Diagnosis / Risk
Degree 1	Linear: $y = \beta_0 + \beta_1 x$	0 bends	Experience vs Salary	Severe **Underfitting** on curved patterns
Degree 2	Quadratic: $y = \beta_0 + \beta_1 x + \beta_2 x^2$	**1 bend** (U or Bell)	• Temp vs HVAC Cost • Fertilizer vs Crop Yield	**Optimal Fit** for natural single-turn phenomena
Degree 3	Cubic: $y = \beta_0 + \ldots + \beta_3 x^3$	2 bends (S-curve)	Technology adoption life-cycle	Marginal gains on simple datasets
Degree 4+	Quartic: $y = \beta_0 + \ldots + \beta_4 x^4$	3+ bends (W-curve)	Complex financial cycles	High **Overfitting** risk (memorizes noise)

1.3 Scikit-Learn Pipeline Implementation (`make_pipeline`)

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

# Pipeline chains feature expansion and OLS regression
model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False), # [x] -> [x, x^2]
    LinearRegression()                                # Fits y = b0 + b1*x + b2*x^2
)
model.fit(X_train, y_train)

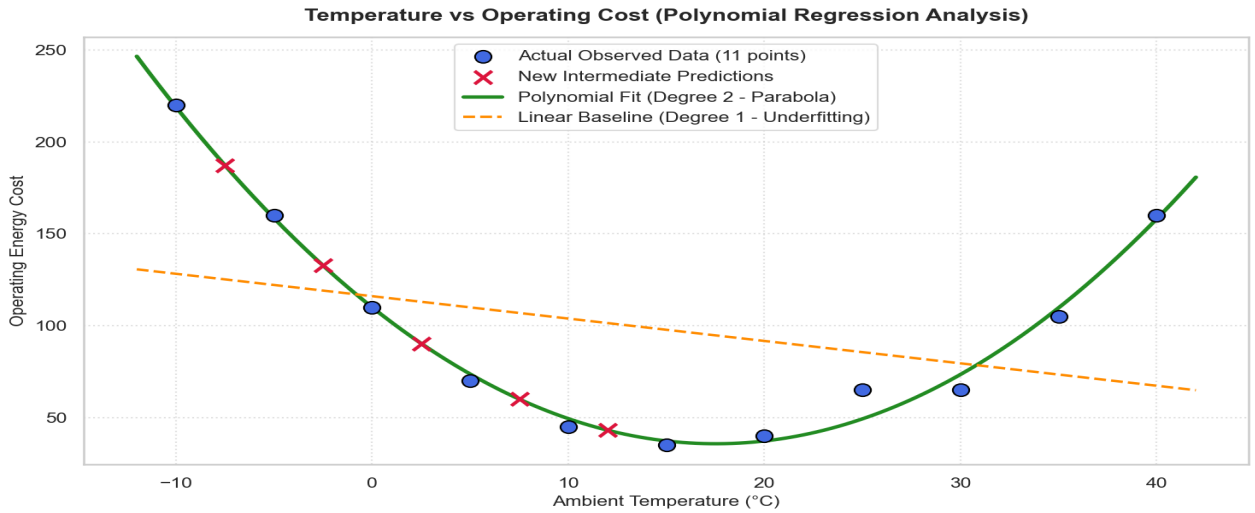
# Extract learned coefficients
lr = model.named_steps["linearregression"]
b0, (b1, b2) = lr.intercept_, lr.coef_
```

Why `include_bias=False`? Setting `include_bias=False` instructs `PolynomialFeatures` not to prepend a column of ones. Because `LinearRegression(fit_intercept=True)` automatically estimates the bias term β_0, leaving `include_bias=True` creates duplicate collinear intercept columns.

1.4 Practical Class Problems Walkthrough

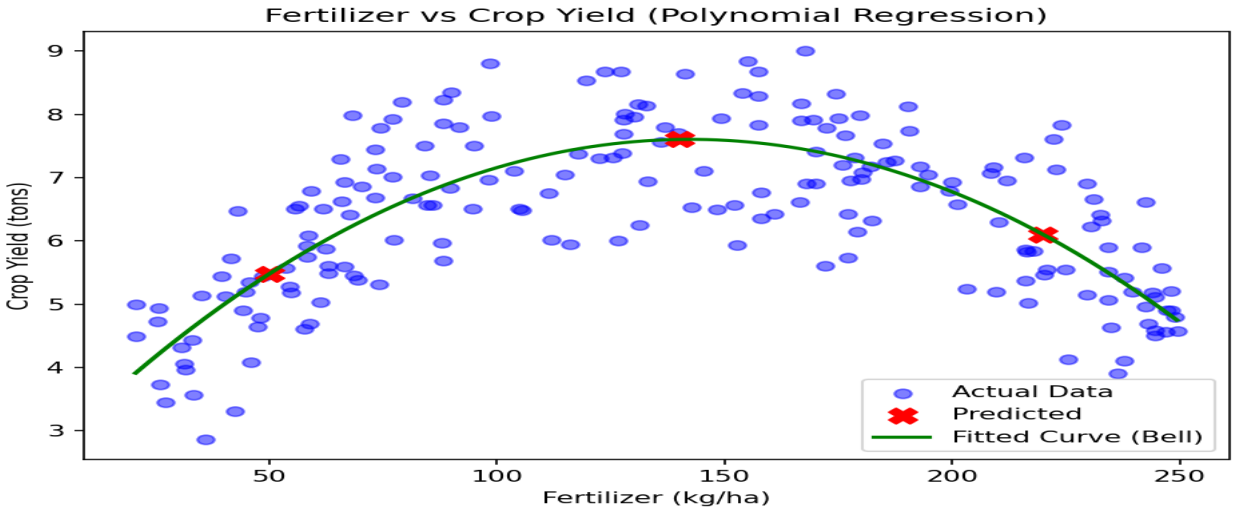
Problem 1: Temperature vs Operating Cost (Parabolic U-Curve)

Facility energy consumption drops as outdoor temperature warms from freezing (-10°C to 0°C), hits minimum in the comfort zone (15°C to 20°C), then climbs back up due to air conditioning load (25°C to 40°C). Fitted Model: **Cost(T) = 109.95 - 8.48·T + 0.24·T^2** (Degree 1 RMSE: 53.76 → Degree 2 RMSE: 5.98).



Problem 2: Fertilizer vs Crop Yield (Bell / Arch Curve)

Agronomic relationship where fertilizer initially accelerates growth, reaches an optimal peak yield at ~142.3 kg/ha, beyond which excess chemical fertilizer causes toxicity/burn and reduces crop yield. Generator: **Yield = 2.0 + 0.08·F - 0.00028·F^2 + Noise**. Fitted Model: **Yield = 2.56 + 0.071·F - 0.00025·F^2** (Degree 1 R^2: 0.0096 → Degree 2 R^2: 0.6013).



PART 2: Unsupervised Learning & K-Means Clustering

2.1 Core Algorithm Mechanics (Lloyd's Algorithm) K-Means is a centroid-based, iterative partitioning algorithm that divides an unlabeled dataset of *N* samples into *K* distinct, non-overlapping clusters. The objective is to minimize within-cluster variance (Inertia).

Phase	Mathematical Operation	Core Logic
1. Initialization	Choose K initial centroid vectors: $\mu_1, \mu_2, ..., \mu_K$	Done randomly or strategically via **K-Means++**
2. Assignment	$c^{(i)} := \text{argmin}_j \|\|x^{(i)} - \mu_j\|\|^2$	Assign each sample to its nearest centroid (Euclidean distance)
3. Update	$\mu_j := (1 / \|C_j\|) \Sigma_{i \in C_j} x^{(i)}$	Recompute centroid as the mean vector of all points in cluster
4. Convergence	$\|\|\mu_j^{(t+1)} - \mu_j^{(t)}\|\| < \text{tol}$	Repeat 2 & 3 until centroids stop moving or `max_iter` reached

2.2 Why K-Means++ is Crucial

Random initialization frequently selects initial centroids in close proximity to each other, leading to slow convergence and trapping the algorithm in poor local minima. **K-Means++** solves this: it selects the first centroid randomly, then selects each subsequent centroid with probability proportional to the squared distance $D(x)^2$ from the nearest existing centroid. This ensures initial centroids are spread widely across the data space.

2.3 Why Feature Scaling (`StandardScaler`) is Mandatory

Because K-Means measures proximity using Euclidean distance (L_2 norm: $d = \sqrt{\Sigma(x_i - y_i)^2}$), features with larger raw numeric magnitudes will completely dominate cluster calculations. For example, if 'Annual Income' ranges from $20,000 to $150,000 and 'Age' ranges from 18 to 70, income will dictate 99.9% of distance computations. Running `StandardScaler()` beforehand guarantees zero mean and unit variance across all dimensions.

2.4 Determining Optimal K (Hyperparameter Tuning)

• **The Elbow Method (Inertia / WCSS):** Plots Inertia ($\Sigma||x - \mu||^2$) against *K*. As *K* increases, Inertia always decreases. Look for the 'elbow' point where the rate of decrease abruptly slows down (diminishing returns).
• **Silhouette Analysis:** Evaluates both within-cluster cohesion (*a*) and between-cluster separation (*b*): $s = (b - a) / max(a, b)$. Ranges from -1 to +1. Peak score indicates optimal compactness and separation.
• **Cluster Profiling:** Always invert-transform fitted centroid coordinates back into original units (`scaler.inverse_transform(kmeans.cluster_centers_)`) to give actionable business labels to clusters.

2.5 Key Limitations of K-Means

• **Assumes Convex / Spherical Clusters:** Partitions space with linear Voronoi boundaries. Fails on concentric rings, moons, or arbitrary density manifolds (use DBSCAN instead).
• **Outlier Sensitivity:** Because centroids compute arithmetic means, extreme outliers distort centroid positions.
• **Fixed K Requirement:** You must specify *K* in advance; the algorithm cannot discover cluster count automatically.

PART 3: Evaluation Metrics Quick Reference Table

Metric	Submodule	Mathematical Formula	Key Insight / Target
MAE	Regression	$(1/n) \Sigma \|y_i - y■_i\|$	Interpretable in target units; robust against extreme outliers.
MSE	Regression	$(1/n) \Sigma (y_i - y■_i)^2$	Quadratic error penalty; heavily penalizes large errors.
RMSE	Regression	$\sqrt{MSE}$	Brings MSE penalty back to target unit scale. Primary regression metric.
R^2 Score	Regression	$1 - (SS_{res} / SS_{tot})$	Variance explained (0 to 1). 1.0 = perfect prediction; 0 = baseline mean.
Adjusted R^2	Regression	$1 - [(1 - R^2)(n-1)/(n-p-1)]$	Penalizes non-informative features added to multiple regression.
Inertia (WCSS)	Clustering	$\Sigma \Sigma \|\|x - \mu_k\|\|^2$	Within-cluster compactness. Lower is better, used in Elbow plot.
Silhouette Score	Clustering	$(b - a) / \max(a, b)$	Range [-1, +1]. High positive value = dense, well-isolated clusters.

PART 4: Key Exam & Viva Multiple-Choice Questions

Q1: Why is feature scaling (e.g., StandardScaler) essential before applying K-Means clustering?

Correct Answer: B — K-Means relies on Euclidean distance, so unscaled features with larger magnitude ranges will disproportionately dominate cluster assignments.

Explanation: K-Means calculates distance vectors using L2 norm. Unscaled features with large variance dictate centroid placement entirely.

Q2: What problem does the K-Means++ algorithm specifically address?

Correct Answer: C — Choosing initial centroid locations strategically to avoid bad local minima and speed up convergence.

Explanation: K-Means++ picks initial centroids probabilistically based on squared distance from existing centroids, preventing cluster collapse.

Q3: What does a Silhouette Score of +0.85 indicate for a clustered dataset?

Correct Answer: B — Clusters are dense, well-separated, and clearly distinct from one another.

Explanation: The Silhouette score ranges from -1 to +1. Values near +1 represent high within-cluster cohesion and strong between-cluster separation.

Q4: Which scenario is K-Means NOT suitable for?

Correct Answer: C — Clustering concentric circle datasets or arbitrary non-convex density shapes.

Explanation: K-Means uses Voronoi partitioning, creating linear decision boundaries. It cannot separate manifold or ring structures (use DBSCAN).